API Petstore REST – visão de negócio

Exemplo de API de uma lojinha de pets, usada para estudar testes e integrações entre sistemas.

O que é / para que serve

- O que é uma API REST
  - Uma interface de programação de aplicações (API) que segue os princípios REST.
  - Usada como um exemplo de estudo para aprender sobre testes e integrações de sistemas.
- Função na Petstore
  - Atua como o "cérebro" por trás das operações da lojinha de pets.
  - Gerencia o estoque, pedidos e informações de usuários.
  - Acessada por outros sistemas, e não diretamente por pessoas.
- Capacidades
  - Permite que sistemas externos realizem ações como:
    - Cadastrar novos pets, pedidos e usuários.
    - Consultar informações existentes sobre pets, pedidos e usuários.
    - Atualizar dados de pets, pedidos e usuários.
    - Remover registros de pets, pedidos e usuários.

Áreas da API (funcionalidades)

- 🐾 Pets
  - Gerenciamento dos animais na loja.
  - **Ações disponíveis:**
    - Cadastrar um novo pet na loja.
    - Buscar pets por status (disponível, pendente, vendido).
    - Ver detalhes de um pet específico (nome, tipo, status).
    - Atualizar informações de um pet existente (mudar nome, status).
    - Remover um pet da loja.
    - Enviar uma imagem do pet para o sistema.
- 🛒 Store (Pedidos)
  - Gerenciamento dos pedidos de compra de pets.
  - **Ações disponíveis:**
    - Consultar o inventário de pets por status (quantidade disponível, vendida, etc.).
    - Criar um pedido de compra de um pet.
    - Ver detalhes de um pedido existente (qual pet, status do pedido).
    - Cancelar um pedido.
- 👤 Users (Usuários)
  - Gerenciamento dos usuários que interagem com o sistema (funcionários, administradores).
  - **Ações disponíveis:**
    - Criar usuários da loja (cadastro individual).
    - Criar uma lista de vários usuários de uma vez.
    - Fazer login de um usuário no sistema (autenticação).
    - Fazer logout (encerrar a sessão do usuário).
    - Ver detalhes de um usuário pelo nome de usuário.
    - Atualizar dados de um usuário existente.
    - Deletar um usuário.

O que ela não faz (limites)

- Não é uma interface de usuário
  - Não possui telas prontas para o cliente final (site ou aplicativo).
  - É um serviço de "bastidores" usado por outros sistemas.
- Não processa pagamentos reais
  - Não lida com transações financeiras de verdade (cartão de crédito, boleto).
  - É apenas um exemplo para simular o fluxo de um pedido.
- Não envia notificações sozinha
  - Não manda e-mails, SMS ou outras notificações automaticamente.
  - Apenas responde às requisições que outros sistemas fazem a ela.
- Não cuida de segurança avançada
  - Não foi projetada para lidar com LGPD ou sistemas de segurança robustos.
  - É um ambiente de testes e estudo.

API de exemplo baseada em Swagger / OpenAPI

- 1. Documentação
  - Swagger UI
    - Interface gráfica interativa para explorar e testar a API.
    - Permite visualizar endpoints, modelos, parâmetros e respostas.
    - Acesso direto para fazer requisições e ver os resultados.
  - OpenAPI Specification (OAS)
    - Padrão para descrever APIs RESTful.
    - Define a estrutura da API em um formato legível por máquina (YAML ou JSON).
    - Facilita a geração de documentação, SDKs e ferramentas de teste.
  - Contrato da API
    - Acordo formal entre o cliente e o servidor sobre o comportamento da API.
    - Define endpoints, métodos HTTP, parâmetros, formatos de requisição/resposta e códigos de status esperados.
  - Schemas / Models
    - Definições da estrutura dos dados utilizados na API.
    - Exemplo: Pet (id, name, category, photoUrls, tags, status), Category (id, name), Tag (id, name), Order (id, petId, quantity, shipDate, status, complete), User (id, username, firstName, lastName, email, password, phone, userStatus).
  - Exemplos de Request e Response
    - Ilustrações de como as requisições devem ser formatadas e as respostas esperadas.
    - Ajuda na compreensão do uso correto de cada endpoint.
- 5. Métodos HTTP
  - GET
    - Consulta de dados.
    - Não deve ter efeitos colaterais no servidor (idempotente).
  - POST
    - Criação de recursos.
    - Envia dados para o servidor para criar um novo recurso.
  - PUT
    - Atualização de recursos.
    - Substitui completamente um recurso existente com os dados fornecidos.
  - DELETE
    - Remoção de recursos.
    - Remove um recurso específico do servidor.
- 6. Status Codes
  - 2xx (Sucesso)
    - 200 OK: Requisição bem-sucedida.
    - 201 Created: Recurso criado com sucesso.
    - 204 No Content: Requisição bem-sucedida, mas sem conteúdo para retornar.
  - 4xx (Erro do Cliente)
    - 400 Bad Request: Requisição inválida, sintaxe incorreta.
    - 401 Unauthorized: Autenticação necessária ou falhou.
    - 403 Forbidden: Acesso negado, mesmo com autenticação.
    - 404 Not Found: Recurso não encontrado.
    - 405 Method Not Allowed: Método HTTP não suportado para o recurso.
  - 5xx (Erro do Servidor)
    - 500 Internal Server Error: Erro genérico no servidor.
    - 502 Bad Gateway: Servidor agindo como gateway ou proxy recebeu uma resposta inválida.
    - 503 Service Unavailable: Servidor temporariamente indisponível.
- 7. Tipos de Testes de API
  - Testes Funcionais
    - Validar se a API atende aos requisitos funcionais.
    - Testar cada endpoint individualmente para verificar se retorna a resposta esperada.
  - Testes Negativos
    - Validar o comportamento da API com entradas inválidas ou cenários de erro.
    - Exemplo: Enviar dados incorretos, IDs inexistentes, credenciais inválidas.
  - Validação de Schema
    - Verificar se as requisições e respostas da API aderem aos schemas definidos na OpenAPI Specification.
    - Garantir a consistência e o formato dos dados.
  - Testes de Contrato
    - Assegurar que a API mantém seu contrato (especificação) ao longo do tempo.
    - Usar ferramentas para comparar o comportamento atual com o contrato esperado.
  - Testes de Integração
    - Verificar a comunicação e o fluxo de dados entre diferentes componentes da API ou com sistemas externos.
    - Testar cenários de ponta a ponta que envolvem múltiplos endpoints.
- 8. Ferramentas de Teste
  - Swagger UI
    - Para exploração manual e testes ad-hoc diretamente na documentação.
  - Postman
    - Ferramenta popular para construção, teste e documentação de APIs.
    - Permite criar coleções de requisições, variáveis de ambiente e scripts de teste.
  - Insomnia
    - Alternativa ao Postman, com foco em simplicidade e design.
    - Oferece recursos semelhantes para testes e gerenciamento de APIs.
  - Newman
    - Executor de linha de comando para coleções do Postman.
    - Ideal para automação e integração em pipelines CI/CD.
  - Rest Assured
    - Biblioteca Java para testar serviços REST.
    - Permite escrever testes legíveis e robustos em Java.
  - Karate DSL
    - Framework de teste de API baseado em Gherkin.
    - Permite escrever testes de forma declarativa e concisa.
  - JMeter
    - Ferramenta para testes de carga e performance.
    - Simula um grande número de usuários acessando a API simultaneamente.
- 9. Autenticação e Segurança
  - API Key
    - Chave única para identificar o cliente que faz a requisição.
    - Geralmente enviada em cabeçalhos (headers) ou parâmetros de query.
  - Headers
    - Utilização de cabeçalhos HTTP para enviar informações de autenticação (ex: Authorization: Bearer <token>).
  - Parâmetros de Autenticação
    - Envio de credenciais (username/password) via corpo da requisição para obter um token de sessão.
  - Controle de Acesso
    - Implementação de autorização para garantir que apenas usuários com permissões adequadas possam acessar determinados recursos.
- 10. Automação e CI/CD
  - Collections Automatizadas
    - Criação de coleções de requisições de teste em ferramentas como Postman.
    - Organização lógica dos testes para execução automatizada.
  - Execução via Newman (CLI)
    - Integração do Newman em scripts de automação.
    - Permite executar coleções de testes de forma programática.
  - Integração com Pipelines CI/CD
    - Inclusão dos testes de API como parte do pipeline de Integração Contínua/Entrega Contínua.
    - Garante que a API seja testada automaticamente a cada nova alteração de código.
    - Exemplo: Jenkins, GitLab CI, GitHub Actions.

Presented with xmind